B.Tech. BCSE497J - Project-I

AN EXPLAINABLE AND DRIFT-AWARE MLOPS PIPELINE FOR CREDIT CARD FRAUD DETECTION

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

23BCE0608 SHAMIK SENGUPTA

23BCE0622 YASHASWINI KANAUJIA

Under the Supervision of

Dr. Sharmila C

Assistant Professor Sr. Grade 1

School of Computer Science and Engineering (SCOPE)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

We hereby declare that the project entitled **An Explainable and Drift-Aware MLOps Pipeline for Credit Card Fraud Detection** submitted by us, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering* to VIT is a record of bonafide work carried out by us under the supervision of Dr. Sharmila C.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date :

Signature of the Candidates

CERTIFICATE

This is to certify that the project entitled **An Explainable and Drift-Aware MLOps Pipeline for Credit Card Fraud Detection** submitted by **Shamik Sengupta (23BCE0608) and Yashaswini Kanaujia (23BCE0622)**, **School of Computer Science and Engineering**, VIT, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering*, is a record of bonafide work carried out by them under my supervision during Fall Semester 2026-2027, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date :

Signature of the Guide

Examiner(s)

Dr. Boominathan P

B.Tech. Computer Science and Engineering

ACKNOWLEDGEMENTS

We are deeply grateful to the management of Vellore Institute of Technology (VIT) for providing us with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in our academic journey. The support and infrastructure provided by VIT have enabled us to explore and develop our ideas to their fullest potential.

Our sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering (SCOPE), for his unwavering support and encouragement. His leadership and vision have greatly inspired us to strive for excellence. The Dean's dedication to academic excellence and innovation has been a constant source of motivation for us. We appreciate his efforts in creating an environment that nurtures creativity and critical thinking.

We express our profound appreciation to Dr. Boominathan P, the Head of the Department, B.Tech. Computer Science and Engineering, for his insightful guidance and continuous support. His expertise and advice have been crucial in shaping the direction of our project. The Head of Department's commitment to fostering a collaborative and supportive atmosphere has greatly enhanced our learning experience. His constructive feedback and encouragement have been invaluable in overcoming challenges and achieving our project goals.

We are immensely thankful to our project guide, Dr. Sharmila C, for her dedicated mentorship and invaluable feedback. Her patience, knowledge, and encouragement have been pivotal in the successful completion of this project. Our supervisor's willingness to share her expertise and provide thoughtful guidance has been instrumental in refining our ideas and methodologies. Her support has not only contributed to the success of this project but has also enriched our overall academic experience.

Thank you all for your contributions and support.

Shamik Sengupta
Yashaswini Kanaujia

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	**x**
1.	**INTRODUCTION**	**1**
	1.1 Background	1
	1.2 Motivations	1
	1.3 Scope of the Project	2
2.	**PROJECT DESCRIPTION AND GOALS**	**3**
	2.1 Literature Review	3
	2.2 Research Gap	4
	2.3 Objectives	5
	2.4 Problem Statement	5
	2.5 Project Plan	5
3.	**TECHNICAL SPECIFICATION**	**7**
	3.1 Requirements	7
	3.1.1 Functional Requirements	7
	3.1.2 Non-Functional Requirements	7
	3.2 Feasibility Study	7
	3.2.1 Technical Feasibility	7
	3.2.2 Economic Feasibility	8
	3.2.3 Social Feasibility	8
	3.3 System Specification	8
	3.3.1 Hardware Specification	8
	3.3.2 Software Specification	9
4.	**DESIGN APPROACH AND DETAILS**	**10**
	4.1 System Architecture	10
	4.2 Design	10
	4.2.1 Data Flow Diagram	10
	4.2.2 Use Case Diagram	11
	4.2.3 Class Diagram	12
	4.2.4 Sequence Diagram	13
5.	**METHODOLOGY AND TESTING**	**15**
	5.1 Module Description	15
	5.1.1 Data Versioning and Splitting	15

5.1.2 Preprocessing Pipeline 15
5.1.3 Model Training and Hyperparameter Tuning 15
5.1.4 Serving API 16
5.1.5 Explainability Module 16
5.1.6 Drift Monitoring Module 17
5.1.7 Containerisation, CI/CD and Deployment 17
5.2 Testing 18
6. PROJECT DEMONSTRATION 20
6.1 Scoring a Fraudulent Transaction 20
6.2 Scoring a Legitimate Transaction and Invalid Input 20
6.3 Batch Prediction 20
6.4 Drift Detection and Alerting 21
7. RESULT AND DISCUSSION 22
7.1 Hyperparameter Search 22
7.2 Holdout Performance 22
7.3 Effect of the Decision Threshold 24
7.4 Explainability Results 25
7.5 Drift Detection Results 25
7.6 Latency 26
7.7 Cost Analysis 27
7.8 Discussion and Limitations 27
8. CONCLUSION 28
8.1 Conclusion 28
8.2 Future Work 28
9. REFERENCES 29
APPENDIX A – SAMPLE CODE 31

List of Figures

Figure No.	Title	Page No.
2.1	Project timeline (July to October 2026)	6
4.1	System architecture	10
4.2	Level-1 data flow diagram	11
4.3	Use case diagram	12
4.4	Class diagram	13
4.5	Sequence diagram for a prediction request	14
7.1	Cross-validated PR-AUC of the 20 Hyperopt trials	22
7.2	Confusion matrix on the holdout set	23
7.3	(a) Precision-recall curve and (b) ROC curve on the holdout set	24
7.4	Global feature importance (mean absolute SHAP value)	25
7.5	Per-feature drift scores for a normal and a shifted window of 300 transactions	26

List of Tables

Table No.	Title	Page No.
2.1	Summary of related work	3
3.1	Hardware specification	8
3.2	Software specification	9
5.1	REST endpoints	16
5.2	Test cases	19
7.1	Best hyperparameters found by Hyperopt	22
7.2	Holdout metrics of the selected model (threshold 0.5)	23
7.3	Comparison with baseline models (holdout)	24
7.4	Precision and recall at different thresholds (holdout, 98 frauds)	25
7.5	Drift check summary	26
7.6	Measured latency	26

List of Abbreviations

API	Application Programming Interface
AUC	Area Under the Curve
CI/CD	Continuous Integration / Continuous Deployment
CPU	Central Processing Unit
CSV	Comma-Separated Values
CV	Cross-Validation
DFD	Data Flow Diagram
DVC	Data Version Control
FN	False Negative
FP	False Positive
GHCR	GitHub Container Registry
HTTP	Hypertext Transfer Protocol
IQR	Interquartile Range
JSON	JavaScript Object Notation
ML	Machine Learning
MLOps	Machine Learning Operations
PCA	Principal Component Analysis
PR-AUC	Area Under the Precision-Recall Curve
RAM	Random Access Memory
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SHAP	SHapley Additive exPlanations
TN	True Negative
TP	True Positive
TPE	Tree-structured Parzen Estimator
TSDB	Time Series Database
UI	User Interface
ULB	Université Libre de Bruxelles
XAI	Explainable Artificial Intelligence
XGBoost	eXtreme Gradient Boosting

Symbols and Notations

y	True class label (1 = fraud, 0 = legitimate)
$\hat{p}$	Predicted probability of fraud
τ	Decision threshold on $\hat{p}$ (default 0.5)
φ_0	SHAP base value (expected model margin, log-odds)
φ_j	SHAP contribution of feature j (log-odds)
w_+	scale_pos_weight, ratio of negatives to positives
W_1	Wasserstein-1 distance used as drift score
s	Drift share, fraction of monitored features flagged
N_w	Drift window size (inputs per evaluation)

ABSTRACT

Card fraud is a very small percentage of the transactions, but every missed fraud costs the bank and the cardholder real money. Creating a classifier that performs well on a notebook is not enough. The model also needs to be served reliably, its decisions need to be explainable to the analyst who reads the flagged payment, and someone needs to detect when incoming transactions deviate from the data it's trained on. This project is the complete pipeline for the public ULB credit card dataset, with 284,807 credit card transactions from Europe, only 492 (0.17%) of which are fraudulent.

We built on the same MLOps template, which was already written for loan approval use case, and restructured it for the fraud detection one. DVC versioning and splitting the data once in code with an 80/20 holdout, while maintaining the same amount of fraud cases in each part. A scikit-learn pipeline (mean imputation, robust scaling of Time and Amount) is used to train an XGBoost classifier using cost sensitive weighting (scale_pos_weight of ~577). Hyperparameters are tuned with Hyperopt's TPE search over 20 trials, where every trial is scored by mean PR-AUC across five stratified folds, and all runs, parameters and model artifacts are tracked in MLflow. We selected PR-AUC to use because accuracy has no meaning here: If a model predicts "legitimate" for all its predictions, then it is already 99.83% accurate.

On the untouched holdout of 56,962 transactions the selected model reaches a PR-AUC of 0.869 and a ROC-AUC of 0.981. With the default threshold at 0.5 it is able to detect 82 of the 98 frauds (recall 0.837) and trigger 13 false alarms (precision 0.863).

A FastAPI app with strict pydantic input validation is used to serve the model. For each fraud transaction, the 5 highest SHAP contributions are returned, exactly as calculated using TreeSHAP. Inference inputs are buffered and each 500 requests are compared to the training distribution by an isolated Evidently worker. The drift share and per-feature scores are exported as Prometheus gauges, alert rules fire when at least 30% of the monitored features have drifted for ten minutes, and a provisioned Grafana dashboard will be used to visualize the drift trend. In a controlled test with 300-transaction windows, normal test traffic showed drift in 2 of 29 features, while deliberately shifted traffic showed drift in 18 of 29 and put both alert rules into the pending state. All of this is deployed with just a single docker compose command. A 16-test regression suite is used to run before building and pushing image via a GitHub Actions workflow, and kubernetes manifests are provided for deployment.

Keywords: credit card fraud, imbalanced classification, XGBoost, SHAP, concept drift, MLOps, MLflow, FastAPI, Prometheus

CHAPTER 1

INTRODUCTION

1.1 Background

Payment in India and across the world has experienced a rapid growth of digital payments in recent ten years, with card not present transactions becoming a common occurrence. As they have grown, so has fraud. The typical fraud transaction is a legitimate one by all surface appearances: it's a typical amount, a typical merchant category, a typical time of day. The signal lies in small irregularities; it's a combination of things a rule written by hand won't pick up. That's why most issuers are now behind each transaction they are willing to approve with a machine learning model, which scores the request and approves it within milliseconds.

In this project, the well-known credit card fraud dataset released by the Machine Learning Group of ULB in collaboration with Worldline [1] is used. It includes transactions by European cardholders for two days in September 2013. Twenty-eight out of the 30 input features (V1-V28) are principal components of the original variables used to protect customer privacy. The elapsed seconds since the first transaction (Time) and the amount (Amount) are stored in raw form. The Class column, which is labeled as fraud, is “1”. The unbalanced nature of the data is what makes it so unique; 492 frauds in 284,807 transactions, or 0.173%.

It is a solved narrow sense problem to train a good model on this data. Gradient-boosted trees such as XGBoost [2] are a strong default, and many notebooks reach high scores. The difficult part, and the one that this project is going to concentrate on, is everything outside of the model. In real ML systems, the model code is a small box in the middle of a large system of data pipelines, configuration, serving and monitoring, and the vast majority of the lifetime costs of the system lie in the surrounding system, Sculley et al. [6] demonstrated. This surrounding system is the domain of MLOps, which is a set of practices that attempts to make it reproducible, testable and observable [7].

1.2 Motivations

This project was brought to us because of three practical issues. First one was the typical approach taken to assessing fraud models in a course or online tutorial. Accuracy is still commonly reported, although it carries very little information at a 0.17% positive rate, and sometimes resampling, like SMOTE [12] is performed prior to the train/test split, thus introducing information into the test set. We desired an evaluation we could justify: a single stratified holdout which is never interacted with during tuning, cross-validation within the training split just, and PR-AUC as the single metric, which Saito and Rehmsmeier [9] suggest for imbalanced problems.

The second gap was explainability. If a fraud analyst is alerted, they should have an understanding of why the model has flagged the payment prior to blocking the card or calling the customer. It is expected of regulators more and more. Just a probability isn't enough to solve that problem. Per transaction analysis of the features that push the score up or down, for tree models it can be computed exactly and quickly [4] and these are SHAP values [3].

Monitoring was the third gap. Fraud patterns evolve as fraudsters evolve and input distribution changes with the seasons and the introduction of new merchants or new payment channels. This is referred to as concept drift by Gama et al. [8]. The model deployed will go bad without anyone realizing it, if the model is never tested against its training distribution. Labels for fraud also come late (usually several weeks after the chargeback), therefore the first signal that is actually available in production is a change in the inputs. We wanted the system to monitor that automatically.

1.3 Scope of the Project

The project is based on one public data set and one model throughout its entire life cycle. It includes data versioning, reproducible splitting, preprocessing, model training with hyperparameter search, experiment tracking, a REST API for single and batch scoring, per-prediction explanations, input drift detection (metrics and alerts), containerisation, a CI pipeline and Kubernetes manifests. It is structured as a series of phases based on the original template: Phase 2: Fraud Detection instead of loan problem, Rebuild training and testing logic; Phase 3a: Add SHAP explanations; Phase 3b: Add drift monitoring; Phase 3c: Package with docker compose, Prometheus, Grafana and DVC.

A few elements are left intentionally out. We do not retrain automatically when a drift is detected since a safe retraining loop requires retraining to be performed on a dataset that includes delayed ground-truth labels, a champion/challenger comparison and a rollback path, none of which can be simulated honestly on a static two-day dataset. We also do not use Alertmanager as an alert receiver, rather we specify alert rules and evaluate them; connecting them to a notification channel is left for future work. Lastly, the features are anonymised PCA components, so we cannot create new behavioural attributes, such as the velocity with which they spend per card.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

The work that is relevant to this project can be grouped into three categories: models for Fraud Detection, Explainability for Tree Ensembles, and the Engineering aspects of deploying and monitoring ML systems.

In the modeling, Bhattacharyya et al. [15] compared the performance of logistic regression, support vector machines and random forests using real card data and concluded that the random forests performed best, and that the results are very sensitive to the sampling of the minority class. Undersampling has been investigated by Dal Pozzolo et al. [1] on the same ULB data and it was demonstrated that it changes the posterior probabilities, which means that the scores have to be recalibrated before they can be used as probabilities. In subsequent research [10] the same authors proposed that realistic evaluation should consider that labels are not immediately available and that an investigator can only investigate a small number of alerts per day, hence their emphasis on precision for the top alerts rather than on accuracy overall. For this problem, Chawla et al. [12] proposed an oversampling method known as SMOTE; however, synthetic oversampling needs to be performed within each training fold to prevent leakage. Scale_pos_weight parameter of XGBoost [2] was described by Chen and Guestrin, which provides a simple cost-sensitive alternative to resampling.

For hyperparameter search, Bergstra et al. [5] proposed a model of good and bad regions of the search space, namely the Hyperopt and Tree-structured Parzen Estimator, which typically requires much fewer trials than grid search. Saito and Rehmsmeier [9] found that on imbalanced data, ROC curves can be very optimistic, since the false positive rate is calculated over a large number of negative examples, and that the precision-recall curve is more representative of the minority class.

For explainability, Lundberg and Lee [3] combined multiple methods into a single framework based on Shapley values in game theory and introduced SHAP. Lundberg et al. [4] then came up with TreeSHAP that computes exact SHAP in polynomial time for tree ensembles, rather than taking a sampling approximation like KernelSHAP.

In the engineering perspective, Sculley et al. [6] talked about how there is hidden technical debt in ML systems, and Breck et al. [13] introduced a rubric of tests for data, model, infrastructure and monitoring. Zaharia et al. [11] proposed MLflow to manage experiments and model packaging. Kreuzberger et al. [7] conducted a survey of MLOps and found the essential elements to be versioning, CI/CD, model registry, serving and monitoring. Paleyes et al. [16] gathered deployment case studies and identified the following as the most common problems: monitoring and model updating. Rabanser et al. [14] tested various approaches to detecting dataset shift and discovered that simple univariate two-sample tests per feature performed surprisingly well. Table 2.1: Papers that influenced our design.

Table 2.1 Summary of related work

Ref.	Work	Key idea	Relevance / gap
[1]	Dal Pozzolo et al., 2015	Undersampling and probability calibration on the ULB dataset	Resampling changes probabilities; we use class weighting instead
[10]	Dal Pozzolo et al., 2018	Realistic fraud setting with delayed labels and alert budgets	Motivates input-drift monitoring since labels arrive late
[2]	Chen and Guestrin, 2016	Scalable gradient boosting (XGBoost)	Base classifier; scale_pos_weight for imbalance
[5]	Bergstra et al., 2013	TPE search in Hyperopt	Tuning method used with stratified CV
[9]	Saito and Rehmsmeier, 2015	PR curve vs ROC curve on imbalanced data	PR-AUC chosen as model-selection metric
[3], [4]	Lundberg et al., 2017, 2020	SHAP and exact TreeSHAP	Per-transaction explanations in the API
[6], [13]	Sculley et al., 2015; Breck et al., 2017	Technical debt and ML test rubric	Single source of truth for features; test gates in CI
[14]	Rabanser et al., 2019	Comparison of dataset-shift detectors	Per-feature statistical drift with a share threshold

2.2 Research Gap

The majority of the published fraud detection literature focuses on offline metrics on a test set. Explainability and drift detection are typically treated as distinct topics and when they are shown, it's usually in a notebook, not in a running service. The initial template we started with had the serving and packaging parts, but the test was hard-coded and failed when the best MLflow run changed, the model was reloaded from MLflow on each request, the serving part pointed to an S3 bucket and tracking server that were no longer in existence, and there was no explanation or monitoring whatsoever.

There is also a practical integration problem which is not often discussed. Popular drift libraries bring in their own web frameworks and dependencies. In our case, Evidently 0.7.x requires a package named multipart, whereas FastAPI 0.103 requires another package called python-multipart, that has the same import name. The two cannot be loaded in one interpreter. The upgrade of the serving stack to make it coexist would have required unpinning of FastAPI and Starlette. Adding monitoring without destabilising the serving path is just the sort of thing that can get you from a demo to a deployable system.

2.3 Objectives

- To create a leak-free training pipeline using a single stratified holdout, stratified cross-validation within the training split and PR-AUC as selection metric on the ULB dataset.
- Tune an XGBoost classifier using Hyperopt and store all trials, parameters, metrics and the final model in MLflow for easy reproducibility.
- To provide the best model via a validated REST API that offers single, form-style and batch CSV predictions.
- To provide a top five contributing features explanation for each fraud flagged prediction, with minimal impact to legitimate traffic latency.
- Expose input drift as Prometheus metrics and trigger alerts if detected automatically against the training distribution without touching the drift library code.
- To wrap up the entire system in Docker, docker compose and Kubernetes manifests, and have a CI process that prevents publishing of images when tests fail.

2.4 Problem Statement

The system is supposed to predict whether a given stream of card transactions is fraudulent or not, given the time of the transaction, the amount of funds involved, and 28 anonymised PCA components describing the transaction. It needs to provide an explanation of the features that led to the identification of a transaction as fraudulent for each transaction it identifies as fraudulent. It should also detect when distribution of incoming transactions deviates from the training distribution and indicate that to those who are operating it. Everything has to be reproducible from versioned data and code, automatically tested and deployable as containers. With less than 2 positives per thousand examples, the model is evaluated based on PR-AUC (precision recall of the fraud class) and not accuracy.

2.5 Project Plan

The work was split into phases, each of which left the system in a working, tested state before the next one started. Fig. 2.1 shows the timeline we followed. The first two weeks went into understanding the baseline template and the dataset. Training and tracking came next, followed by the test suite and CI, so that every later change was checked automatically. Explainability (Phase 3a) and drift monitoring (Phase 3b) were added on top of a stable serving path, and Phase 3c packaged the services together. Shamik Sengupta led the training pipeline, Docker and CI work, and Yashaswini Kanaujia led the explainability, drift monitoring and observability work; both of us shared testing, evaluation and documentation.

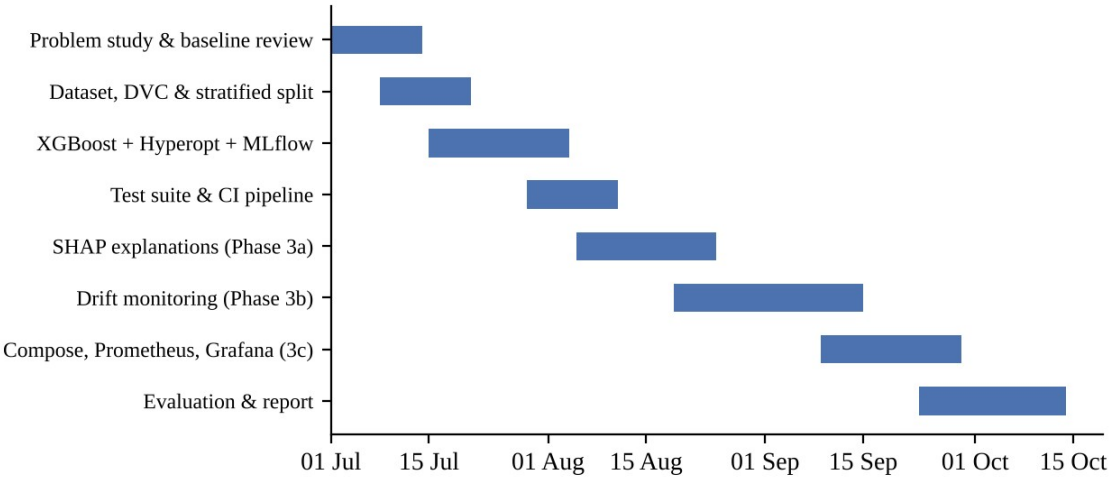


Fig. 2.1 Project timeline (July to October 2026)

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional Requirements

- FR1: The system shall accept a single transaction (Time, V1 to V28, Amount) as JSON and return a label (Fraud or Legitimate) and the fraud probability.
- FR2: For a transaction labelled Fraud, the response shall include the base value and the five features with the largest absolute SHAP contribution, each with its raw value, SHAP value and direction.
- FR3: The system shall reject requests with missing or non-numeric fields with HTTP 422 before they reach the model.
- FR4: The system shall accept a CSV file for batch scoring, append a Prediction column, return the file and archive a copy by date.
- FR5: The training pipeline shall tune the model, log all trials and the final model to MLflow, and the API shall always serve the run with the highest holdout PR-AUC.
- FR6: The system shall buffer inference inputs and recompute drift against the training reference every N_w inputs, and also on demand through an endpoint.
- FR7: Drift results shall be exposed as Prometheus metrics and evaluated by alert rules; a Grafana dashboard shall display them.

3.1.2 Non-Functional Requirements

- Performance: single-transaction scoring, including the explanation when one is produced, should complete well within 100 ms end to end on commodity hardware.
- Reliability: a failure in drift computation must never fail or slow down a prediction request.
- Reproducibility: data is pinned by DVC hash, the split and all searches are seeded (random_state = 42), and the core library versions are pinned in requirements.txt (prometheus-fastapi-instrumentator is not yet pinned).
- Maintainability: the feature list, preprocessing steps and split logic each exist in exactly one place in the code.
- Portability: the service runs identically on the developer machine, inside docker compose and on Kubernetes.
- Testability: unit tests must run offline without the raw dataset or an MLflow server, so they can run in CI and during the Docker build.

3.2 Feasibility Study

3.2.1 Technical Feasibility

All components are mature open-source tools with good documentation. The dataset is small enough (about 150 MB) to train on a laptop: the full search of 20 trials with five folds each,

plus the final refit, took about 6.4 minutes on our machine according to the MLflow run timestamps. TreeSHAP on a 300-tree model of depth 5 explains a single row in roughly 12 ms. The only real technical risk was the dependency conflict between Evidently and FastAPI described in Section 2.2, which we resolved by running the drift computation in a separate worker process.

3.2.2 Economic Feasibility

Every tool in the stack (Python, scikit-learn, XGBoost, SHAP, MLflow, Hyperopt, FastAPI, Evidently, Prometheus, Grafana, Docker, DVC and GitHub Actions on public repositories) is free to use, so there is no licensing cost. Training runs on a CPU and does not need a GPU. The trained model is 515 KB, and the serving container can run on a small instance. For a bank, even a handful of extra frauds caught per day would outweigh the cost of hosting such a service.

3.2.3 Social Feasibility

Fraud detection protects cardholders from financial loss and reduces the stress of disputed transactions. Explanations make it easier for an analyst to justify a decision to a customer and reduce the chance of blocking a genuine card for reasons nobody can explain. The dataset is anonymised through PCA, so no personal information is exposed in this work. A real deployment would still need to consider fairness across customer groups, which is not possible to measure on this dataset because demographic attributes are not available.

3.3 System Specification

3.3.1 Hardware Specification

Table 3.1 Hardware specification

Component	Minimum	Used for development
Processor	4-core x86-64 CPU	HP Victus laptop, multi-core x86-64 CPU
Memory	8 GB RAM	16 GB RAM
Storage	5 GB free (dataset, venv, images)	SSD
GPU	Not required	Not used
Network	Needed only for pip, Docker images and CI	Broadband

3.3.2 Software Specification

Table 3.2 Software specification

Category	Tool / library	Version
Language	Python	3.11 (local venv), 3.10 (Docker image)
ML	scikit-learn, XGBoost, NumPy, pandas	1.2.2, 2.0.3, 1.24.3, 1.5.3
Tuning / tracking	Hyperopt, MLflow	0.2.7, 2.13.2
Explainability	SHAP (TreeExplainer)	0.44.1
Serving	FastAPI, pydantic, Uvicorn	0.103.0, 2.4.2, 0.23.2
Drift	Evidently (isolated worker)	0.7.21
Observability	prometheus-fastapi-instrumentator, Prometheus, Grafana	latest, 2.53.0, 11.1.0
Versioning / CI	Git, DVC, GitHub Actions	DVC 3.51.2
Deployment	Docker, docker compose, Kubernetes (Docker Desktop)	-
Testing / IDE	pytest, VS Code	7.4.2
Operating system	Windows 11 (development), Linux containers	-

CHAPTER 4

DESIGN APPROACH AND DETAILS

4.1 System Architecture

Fig. 4.1 shows the overall architecture. It has four parts. The offline part runs on the developer machine: the DVC-tracked dataset is split, the model is tuned and trained, and the run is logged to MLflow; the same deterministic split is also used to produce the drift reference file. The CI part runs on GitHub: every push runs the test suite and, only if it passes, builds the Docker image and pushes it to the GitHub Container Registry. The serving part is the FastAPI application, which loads the best MLflow run, keeps the model and its SHAP explainer in memory, and records each inference input in a rolling buffer. The observability part is Prometheus, which scrapes the /metrics endpoint every 15 seconds and evaluates the alert rules, and Grafana, which reads from Prometheus.

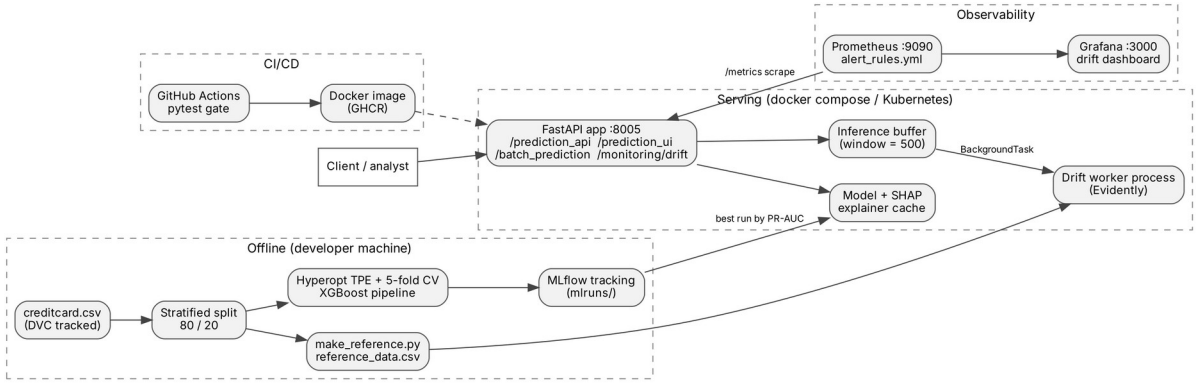


Fig. 4.1 System architecture

Two design decisions deserve a short explanation. First, the model is not reloaded on every request. The API resolves the best run ID, caches it for 60 seconds and refreshes it in the background, and keeps the loaded model and explainer in a cache keyed on that ID, so they are rebuilt only when a better run appears. Second, the drift computation never runs inside the API process. The API only appends rows to a buffer and, when a window fills, schedules a FastAPI BackgroundTask that launches drift_worker.py as a separate Python process. The worker imports Evidently, compares the window with the reference, writes a small JSON summary and exits. The API then reads that summary and updates the gauges. This keeps the incompatible multipart packages in different interpreters and keeps the drift work off the request latency path.

4.2 Design

4.2.1 Data Flow Diagram

Fig. 4.2 is a level-1 data flow diagram. A transaction enters process 1.0, where it is validated against the typed schema. Invalid input goes straight back to the client with a 422 response. Valid input goes to process 2.0, which loads the model artifact from the MLflow store (D1) and scores it. Only fraud-flagged transactions continue to process 3.0 for explanation. In parallel, the raw input is written to the inference buffer (D2). Process 4.0 reads the buffer and

the reference data (D3) and writes drift gauges into Prometheus (D4), from where the analyst sees alerts and the dashboard. Process 5.0 is the offline training flow that reads the raw dataset (D5) and writes parameters, metrics and the model into D1.

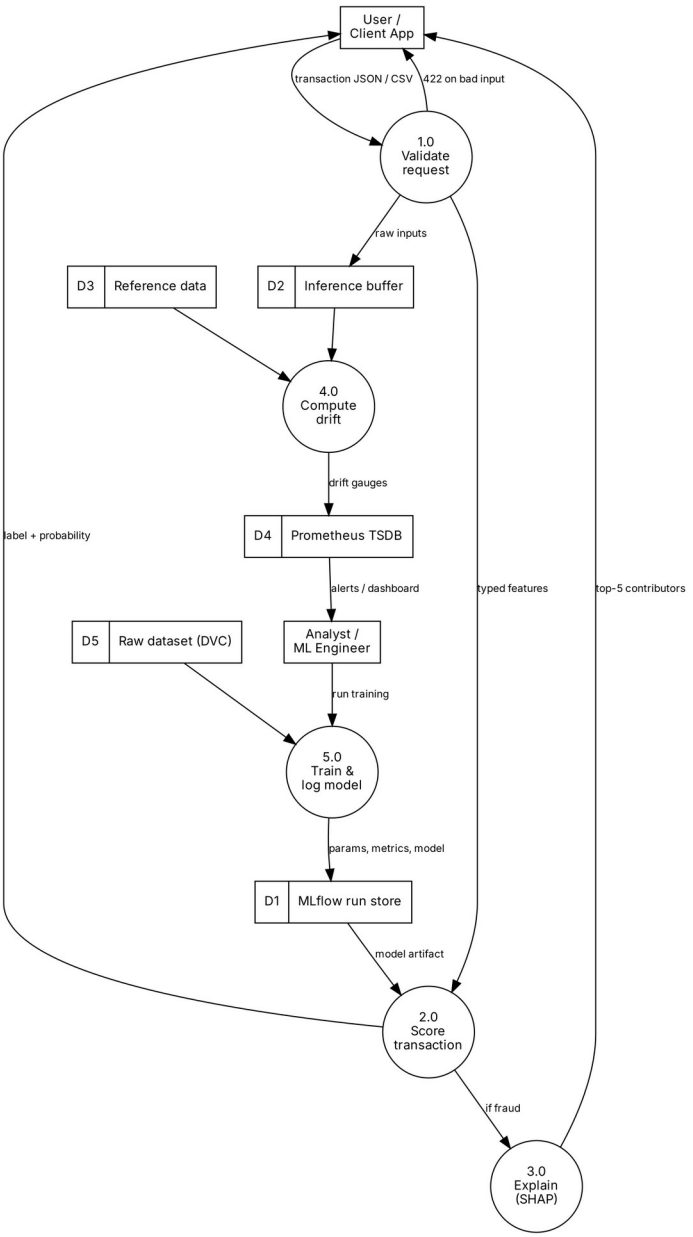


Fig. 4.2 Level-1 data flow diagram

4.2.2 Use Case Diagram

There are three kinds of users (Fig. 4.3). A client system or bank analyst scores transactions, reads the explanation and uploads batch files. The ML engineer trains and tunes models, compares runs in MLflow, builds and deploys the image and can trigger a drift check manually. An operations or on-call person watches the dashboard and responds to drift alerts.

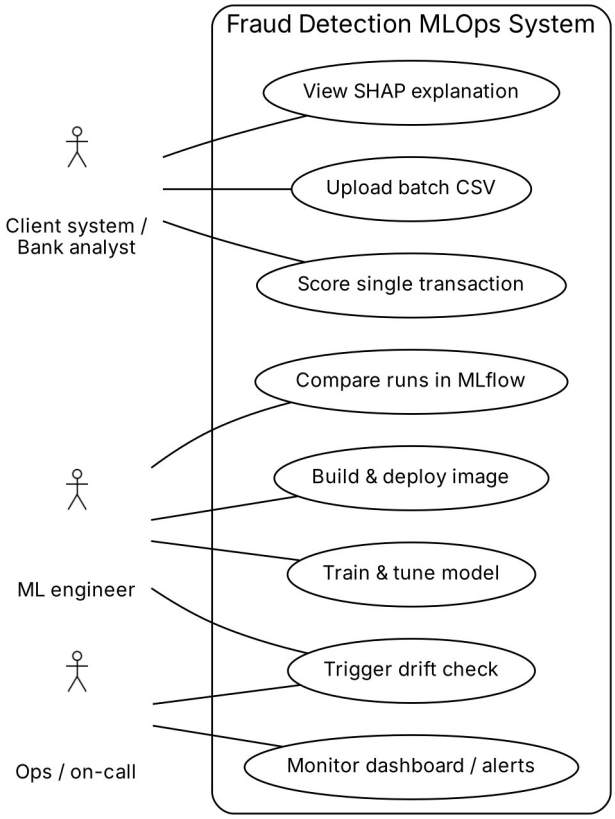


Fig. 4.3 Use case diagram

4.2.3 Class Diagram

The codebase is mostly functional, but the main types and modules are shown in Fig. 4.4. The two custom transformers inherit from scikit-learn's BaseEstimator and TransformerMixin so that they can be fitted inside a Pipeline and cross-validated without leakage. The request and response models are pydantic classes. TransactionFeatures is generated from the feature list with create_model, so the schema cannot drift away from config.FEATURES. PredictionResponse nests an optional PredictionExplanation, which holds a list of FeatureContribution objects. The predict and drift modules hold module-level state (the model cache and the rolling buffer).

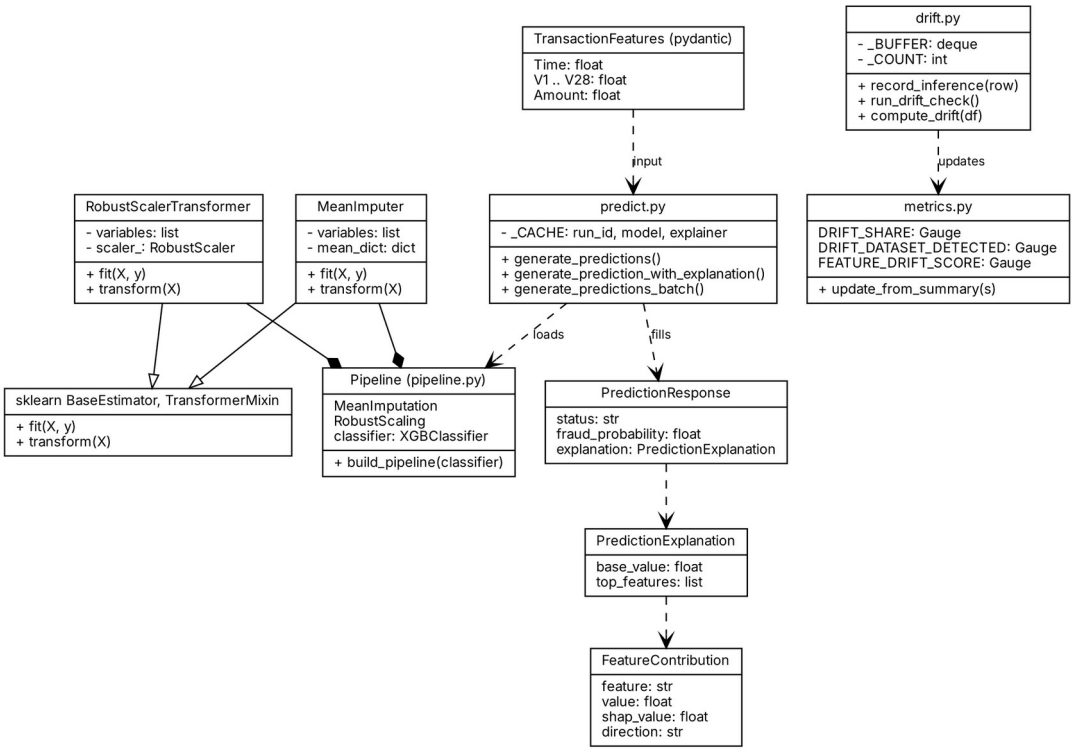


Fig. 4.4 Class diagram

4.2.4 Sequence Diagram

Fig. 4.5 traces one request to /prediction_api. After validation, main.py calls predict.py, which asks MLflow for the best run ID and loads the model only on a cache miss. The probability and label are computed, and SHAP is run only if the label is fraud. The input is then recorded in the drift buffer, and the response is returned to the client. If that request completed a window, a background task runs the drift check after the response has already been sent, so the client never waits for it.

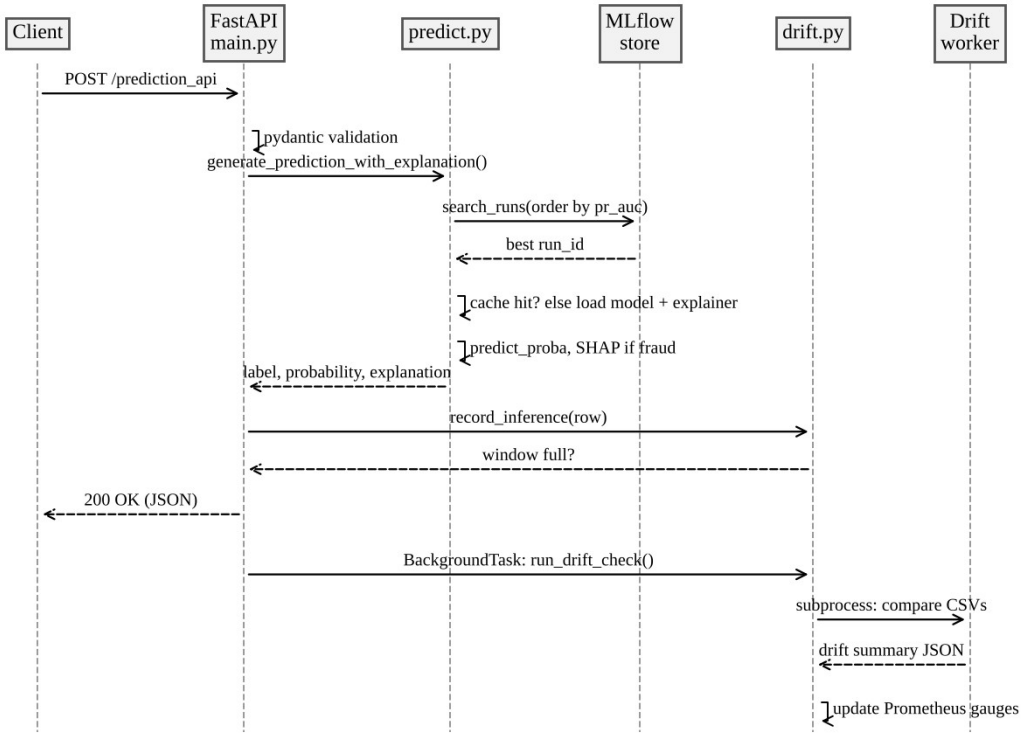


Fig. 4.5 Sequence diagram for a prediction request

CHAPTER 5

METHODOLOGY AND TESTING

5.1 Module Description

5.1.1 Data Versioning and Splitting

It is not possible to store the raw file creditcard.csv in Git because it is too large. It is stored with DVC instead: The repository only stores creditcard.csv.dvc, which contains the file's MD5 hash (e90efcb8...) and size, and the actual file is stored in a DVC remote. The hash pins the exact file; with a shared remote, dvc pull would fetch the same bytes. We had to add a commit of pinning pathspec to 0.11.2 to requirements-dev.txt as newer versions of pathspec removed a private symbol that DVC 3.51 imports.

The split is done in code by stratified_split() in processing/data_handling.py, which is the only place a train/test split is made. It splits the data using train_test_split with test_size = 0.2, random_state = 42 and stratify = y. This results in 227,845 training rows, 394 frauds, and 56,962 test rows, 98 frauds, keeping the 0.17% fraud rate in both halves. The holdout is used only once, at the end of tuning. All the cross validation takes place within the training split.

5.1.2 Preprocessing Pipeline

V1 to V28 are already PCA outputs, centred and roughly scaled, so they are passed through unchanged. Time and Amount are raw and heavily skewed; Amount in particular has extreme outliers. We scale these two columns with a RobustScaler, which uses the median and interquartile range, so a few very large transactions do not dominate the scaling the way they would with min-max or standard scaling. A MeanImputer runs first as a safety net: the dataset has no missing values, but a batch CSV uploaded later might. Both transformers are defined in processing/preprocessing.py and assembled by build_pipeline() in pipeline.py, which is the single definition used by training, testing and serving. Because they are fitted inside the Pipeline, cross_val_score refits them on each training fold and they never see the rows they are evaluated on.

5.1.3 Model Training and Hyperparameter Tuning

The classifier is XGBoost's XGBClassifier. Instead of resampling, class imbalance is handled with scale_pos_weight, set to the ratio of legitimate to fraudulent transactions in the training split (227,451 / 394, which is about 577). This tells the booster to weight each fraud as heavily as about 577 legitimate transactions in the loss. The evaluation metric is set to aucpr.

Hyperopt's TPE algorithm searches nine hyperparameters: max_depth (3 to 9), learning_rate (0.01 to 0.3), n_estimators (100 to 450 in steps of 50), subsample and colsample_bytree (0.5 to 1.0), gamma (0 to 5), reg_alpha and reg_lambda (0 to 1) and max_delta_step (0 to 10). The last one is recommended for logistic objectives on very imbalanced data because it limits how large each update can be. For every trial, the objective function builds a fresh pipeline, runs 5-fold StratifiedKFold cross-validation on the training split with average_precision as the score, logs the parameters and the mean and standard deviation of the fold scores to a nested MLflow run, and returns the negative mean PR-AUC for Hyperopt to minimise. After 20 trials, the best

configuration is refitted on the whole training split and evaluated once on the holdout. The parent MLflow run stores the best parameters, the holdout PR-AUC, ROC-AUC, precision, recall and F1, and the fitted pipeline as an MLflow sklearn model.

```
def objective(params):
    clf = make_classifier(params)
    classification_pipeline = pipe.build_pipeline(clf)
    skf = StratifiedKFold(n_splits=config.CV_FOLDS, shuffle=True,
                          random_state=config.RANDOM_STATE)
    scores = cross_val_score(classification_pipeline, X_train, y_train,
                             cv=skf, scoring='average_precision', n_jobs=1)
    with mlflow.start_run(nested=True):
        mlflow.log_params({...})
        mlflow.log_metric('cv_pr_auc_mean', float(scores.mean()))
    return {'loss': -float(scores.mean()), 'status': STATUS_OK}
```

5.1.4 Serving API

main.py exposes the endpoints listed in Table 5.1. The request body for the prediction endpoints is TransactionFeatures, in which all 30 fields are required floats. This typed boundary replaced the baseline's untyped string fields: anything malformed is rejected by FastAPI with a 422 response that names the offending field, before any model code runs. The service always uses the MLflow run with the highest holdout PR-AUC, found with mlflow.search_runs ordered by metrics.pr_auc. CORS is enabled and prometheus-fastapi-instrumentator adds request count and latency histograms on /metrics. Batch outputs are archived under batch_predictions/<date>/<file>_<timestamp>.csv; the timestamp uses hyphens instead of colons because colons are not allowed in Windows file names, which we found out the hard way.

Table 5.1 REST endpoints

Method	Endpoint	Purpose
GET	/	Health / welcome message
POST	/prediction_api	Label, fraud probability and SHAP explanation (if fraud)
POST	/prediction_ui	Label only, for simple form-based clients
POST	/batch_prediction	Upload CSV, returns CSV with a Prediction column
POST	/monitoring/drift	Run a drift check now on the buffered inputs
GET	/monitoring/drift	Return the last drift summary
GET	/metrics	Prometheus metrics (HTTP and drift gauges)
GET	/docs	Interactive Swagger documentation

5.1.5 Explainability Module

explain.py wraps SHAP's TreeExplainer around the XGBoost step of the pipeline (model.named_steps['classifier']). We chose TreeExplainer over KernelExplainer because XGBoost is a tree ensemble, and TreeSHAP gives exact Shapley values in polynomial time, whereas KernelExplainer approximates them by perturbing a background sample and is orders of magnitude slower. Since the explainer sees only the classifier, each row is first passed through model[:-1], the preprocessing steps, so that the explainer works in the same feature space as the model. Both transformers keep the DataFrame column names, so the explanation still refers to Time, V1 to V28 and Amount.

The SHAP values are in log-odds (margin) units, not probability. The base value plus the sum

of all 30 SHAP values equals the model's raw margin for that row. We return the five features with the largest absolute contribution, sorted, together with the raw value the client sent and a direction field (increases_fraud_risk or decreases_fraud_risk). The units are written into the pydantic field descriptions, so they show up directly in the Swagger documentation. Explanations are attached only to fraud-flagged transactions (EXPLAIN_ONLY_FLAGGED = True), so the extra cost scales with the 0.17% flag rate and not with total traffic. Different SHAP versions return values as a list, a 3-D array or a 2-D array; the helper _positive_class_shap() normalises all three, and a unit test pins the shape we actually get.

5.1.6 Drift Monitoring Module

The monitoring package is split into four files so that the serving process never imports Evidently. metrics.py defines the Prometheus gauges and imports nothing except prometheus_client; it is imported at application start-up so that the drift series appear on /metrics from the first scrape. drift.py holds a thread-safe deque of the most recent N_w inputs (N_w = 500 by default, configurable through the DRIFT_WINDOW environment variable). record_inference() keeps only the monitored features and returns True every N_w inputs, which makes main.py schedule a background check. drift_worker.py is a standalone script that runs in its own interpreter. Before importing Evidently it loads the standalone multipart.py module directly into sys.modules so that litestar finds the package it expects. It then runs Evidently's DataDriftPreset on the shared columns and reduces the result to a compact JSON summary. make_reference.py regenerates the reference by re-running the same seeded split and sampling 20,000 rows of X_train.

Twenty-nine features are monitored: V1 to V28 and Amount. Time is excluded on purpose. It counts seconds since the first transaction in the dataset, so any live data will always lie beyond the training range and would show permanent drift that means nothing. For numerical columns Evidently uses a normed Wasserstein distance with a threshold of 0.1. The drift share s is the fraction of the 29 features above that threshold, and dataset drift is declared when s is at least 0.3. Five gauges are exported for the dataset as a whole (detected flag, share, drifted column count, window size, last run timestamp) and one labelled gauge with the score of each feature. Every feature is set on every run, so a feature that stops drifting falls back on the dashboard instead of keeping an old high value. Two alert rules in alert_rules.yml use these gauges, FraudInputDatasetDrift and FraudInputDriftShareHigh, each held for 10 minutes so that a single noisy window does not trigger them. FraudInputDatasetDrift fires on the dataset flag ($s \geq 0.3$), while FraudInputDriftShareHigh uses a strict $s > 0.3$, so the two differ only when s is exactly 0.3.

5.1.7 Containerisation, CI/CD and Deployment

The Dockerfile starts from python:3.10-slim-bookworm, installs the pinned requirements, copies the code, and runs pytest during the build, so an image cannot be built from code that fails its tests. docker-compose.yml starts three services: the app on port 8005, Prometheus 2.53.0 on 9090 with the scrape configuration and alert rules mounted read-only, and Grafana 11.1.0 on 3000 with the datasource and dashboard provisioned from files. The mlruns directory and the reference file are bind-mounted into the app container. We ran into two Docker problems here. When the reference file did not exist, Docker silently created a directory with that name, and nesting a file mount inside a mounted directory caused Docker

to write an empty placeholder back into the repository. The compose file now mounts every target as a separate leaf path, with comments explaining why.

The GitHub Actions workflow in .github/workflows/main.yml has two jobs. The test job installs the requirements on Python 3.10 and runs pytest on every push and pull request to main. The build-and-push job depends on it and runs only on pushes: it logs in to GHCR with the workflow token and pushes the image tagged both latest and with the commit SHA. For Kubernetes, deployment.yml runs one replica of the image with the mlruns folder mounted from the host, and service.yml exposes it through a LoadBalancer on port 80. The manifests are written for the Kubernetes cluster that ships with Docker Desktop.

5.2 Testing

The test suite follows the idea from Breck et al. [13] that data and model behaviour should be tested, not just code paths. The baseline had one test that checked a hard-coded prediction and broke whenever the best MLflow run changed, so it was never a real gate. We replaced it with 16 tests in two files that run offline, without the raw dataset or an MLflow server, using synthetic data where a dataset is needed. This is what allows them to run in CI and inside the Docker build. Table 5.2 lists them. The full suite passes in about 15 seconds.

Table 5.2 Test cases

No.	Test	What it checks	Result
1	test_schema_accepts_valid_row	A complete row of 30 floats is accepted	Pass
2	test_schema_rejects_missing_field	Dropping Amount raises ValidationError	Pass
3	test_schema_rejects_wrong_type	Amount = "not_a_number" is rejected	Pass
4	test_feature_contract	30 features, Time first, Amount last, schema fields = config.FEATURES	Pass
5	test_split_preserves_class_ratio	Fraud rate in train and test within 0.3% of the full rate	Pass
6	test_split_is_deterministic	Two splits give identical indices	Pass
7	test_explain_row_shape_and_ranking	Top-N length, sorted by |SHAP|, correct keys and direction	Pass
8	test_explain_row_respects_top_n	top_n = 3 returns exactly three features	Pass
9	test_record_inference_fires_every_window	With window 5, fires at the 5th and 10th input only	Pass
10	test_record_inference_keeps_only_monitored_features	Time is dropped from the buffer	Pass
11	test_summarize_extracts_count_share_and_scores	Parser reads count, share and per-feature scores	Pass
12	test_summarize_no_drift_below_threshold	No dataset drift when all scores are low	Pass
13	test_summarize_fills_missing_features_with_zero	Missing features reported as 0.0	Pass
14	test_update_from_summary_sets_gauges	Gauge values readable from the Prometheus registry	Pass
15	test_worker_end_to_end_detects_broad_shift	Real worker process flags drift when 15 of 29 columns shift	Pass (Docker/CI, Python 3.10)
16	test_best_run_id_is_cached_and_refreshed_in_background	Best run looked up once, served from cache, refreshed in background after expiry	Pass

Beyond the automated tests, we tested the running service by hand. We sent valid fraud and legitimate transactions from the test split, sent a request with a string in the Amount field to confirm the 422 response, uploaded a 200-row CSV to the batch endpoint, and pushed normal and deliberately shifted traffic through the API to check that the drift gauges and alerts responded. These results are presented in Chapters 6 and 7.

CHAPTER 6

PROJECT DEMONSTRATION

This chapter walks through the running system. The stack is started with two preparation commands followed by docker compose up:

```
# 1. write reference_data.csv
python -m prediction_model.monitoring.make_reference
# 2. make the mlruns paths portable between host and container
python -m prediction_model.fix_mlruns_paths
# 3. app :8005, Prometheus :9090, Grafana :3000
docker compose up --build
```

6.1 Scoring a Fraudulent Transaction

We took a transaction from the test split that is labelled as fraud and posted it to /prediction_api. The model returned a fraud probability of 0.99997 and attached its explanation. The trimmed response is shown below. V14, V10 and V12 had strongly negative values and were the three strongest reasons for the fraud label, while V8 and V2 pulled the score slightly back towards legitimate.

```
POST /prediction_api
{
  "status": "Fraud",
  "fraud_probability": 0.9999656677246094,
  "explanation": {
    "base_value": 8.5388,
    "top_features": [
      {"feature": "V14", "value": -6.174, "shap_value": 3.117,
       "direction": "increases_fraud_risk"},
      {"feature": "V10", "value": -4.881, "shap_value": 1.746,
       "direction": "increases_fraud_risk"},
      {"feature": "V12", "value": -4.686, "shap_value": 1.234,
       "direction": "increases_fraud_risk"},
      {"feature": "V8", "value": 1.167, "shap_value": -0.922,
       "direction": "decreases_fraud_risk"},
      {"feature": "V2", "value": 2.463, "shap_value": -0.687,
       "direction": "decreases_fraud_risk"}
    ]
  }
}
```

6.2 Scoring a Legitimate Transaction and Invalid Input

A legitimate transaction from the test split returned status Legitimate with a probability of 2.47×10^{-6} and explanation set to null, since no explanation is computed for non-flagged traffic. When we changed Amount to the string "abc", the request never reached the model. FastAPI returned HTTP 422 with the following body:

```
{"detail": [{"type": "float_parsing",
             "loc": ["body", "Amount"],
             "msg": "Input should be a valid number, unable to parse
                     string as a number",
             "input": "abc"}]}
```

6.3 Batch Prediction

We uploaded the first 200 rows of the test split as sample_batch.csv to /batch_prediction. The service returned the same file with a Prediction column (fraud or legit) appended, as a CSV download, and wrote a copy to batch_predictions/2026-10-14/. Extra columns such as the ground-truth Class column are ignored, because the input is sliced to config.FEATURES

before prediction. These first 200 rows contain no fraud, so the web console uses a different 200-row sample (185 legitimate and 15 fraud), in which the model caught 14 of the 15 frauds with no false alarms.

6.4 Drift Detection and Alerting

To demonstrate monitoring, we ran the API with DRIFT_WINDOW = 300 and sent 300 real test transactions. The drift check flagged only 2 of 29 features (s = 0.069), so no dataset drift was reported. We then sent 300 more test transactions after modifying them to imitate a change in customer behaviour: Amount was multiplied by 3 with 50 added, and twelve PCA components (V1 to V7, V9 to V12 and V14) were shifted by 1.5. This time 18 of 29 features were flagged (s = 0.621), dataset_drift_detected became 1, and the /metrics endpoint exposed the new values:

```
fraud_input_drift_dataset_detected 1.0
fraud_input_drift_share 0.6206896551724138
fraud_input_drifted_columns_count 18.0
fraud_input_drift_window_size 300.0
fraud_input_feature_drift_score{feature="V7"} 1.3053805446288955
```

Prometheus picked up the gauge on its next scrape, and both alert rules moved into the pending state. They would move to firing if the condition held for the configured 10 minutes. The Grafana dashboard provisioned from grafana_dashboard.json has four panels: the drift share over time with the 0.3 threshold, a stat panel for the dataset drift flag, a stat panel for drifted columns and window size, and a time series of the per-feature drift scores.

CHAPTER 7

RESULT AND DISCUSSION

7.1 Hyperparameter Search

Fig. 7.1 shows the mean cross-validated PR-AUC of the 20 Hyperopt trials, sorted from worst to best. Scores ranged from 0.773 to 0.854 with a mean of 0.836. Most trials were packed between 0.84 and 0.85, so a wide range of settings performs about equally well. Default XGBoost, with no tuning at all, reaches the same cross-validated score (0.854), so the eight-point spread reflects a few poor settings rather than a gain from tuning. The best configuration is listed in Table 7.1. It is a fairly shallow model (depth 5) with 300 trees, a moderate learning rate and quite strong row subsampling (0.64), which helps with variance given that there are only 394 frauds to learn from.

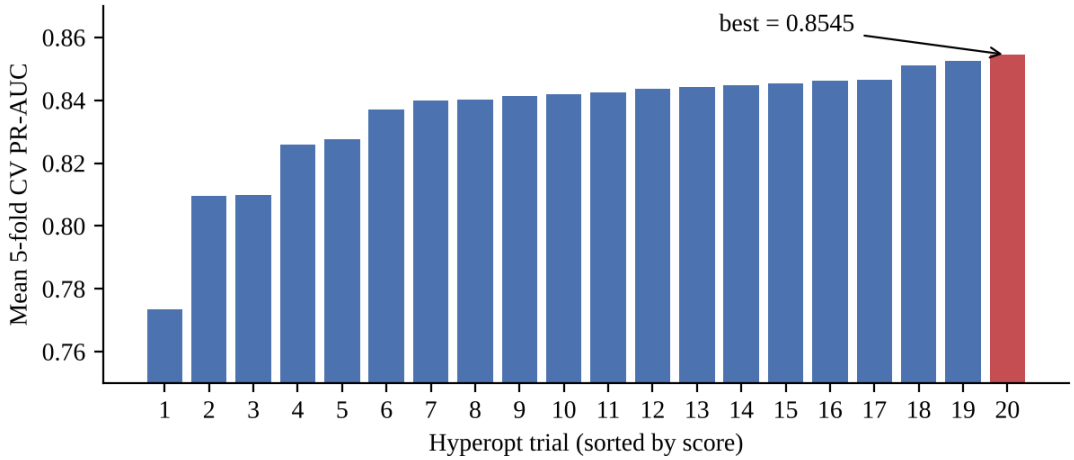


Fig. 7.1 Cross-validated PR-AUC of the 20 Hyperopt trials

Table 7.1 Best hyperparameters found by Hyperopt

Parameter	Value	Parameter	Value
max_depth	5	gamma	3.164
n_estimators	300	reg_alpha	0.033
learning_rate	0.093	reg_lambda	0.860
subsample	0.636	max_delta_step	0
colsample_bytree	0.829	scale_pos_weight	577.3

7.2 Holdout Performance

Table 7.2 gives the results of the refitted model on the 56,962-row holdout. These are the values logged to MLflow, and we reproduced them exactly by reloading the model and re-running the split. The holdout PR-AUC (0.869) is slightly higher than the best cross-validated score (0.854). The difference is within the fold-to-fold variation and is expected, since the final model is trained on all of the training data instead of four-fifths of it.

Table 7.2 Holdout metrics of the selected model (threshold 0.5)

Metric	Value
PR-AUC (average precision)	0.8693
ROC-AUC	0.9809
Precision (fraud)	0.8632
Recall (fraud)	0.8367
F1-score (fraud)	0.8497
Accuracy	0.9995
Accuracy of always predicting legitimate	0.9983

The last two rows show why we do not report accuracy as a result. Our model's accuracy is 99.95%, but a model that never predicts fraud already gets 99.83%. The confusion matrix in Fig. 7.2 is more informative: of the 98 frauds in the holdout, 82 were caught and 16 were missed, while 13 of the 56,864 legitimate transactions were wrongly flagged. In practical terms an analyst would need to review 95 alerts to find 82 frauds, which is a manageable workload.

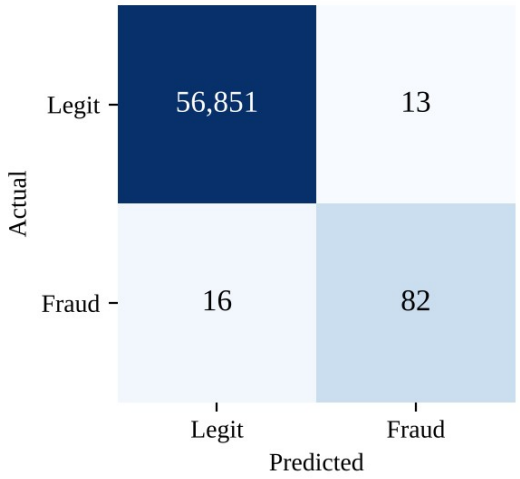


Fig. 7.2 Confusion matrix on the holdout set

Fig. 7.3 shows the precision-recall and ROC curves. The dashed line in the PR plot is the precision of a random classifier, which equals the fraud rate (0.0017). The ROC curve rises almost vertically and its AUC is 0.981, but this mostly reflects how easy it is to keep the false positive rate low when there are 56,864 negatives. The PR curve is the more honest picture: precision stays above 0.8 up to a recall of roughly 0.8 and then falls quickly, because the last 15 to 20 percent of frauds look very similar to legitimate transactions.

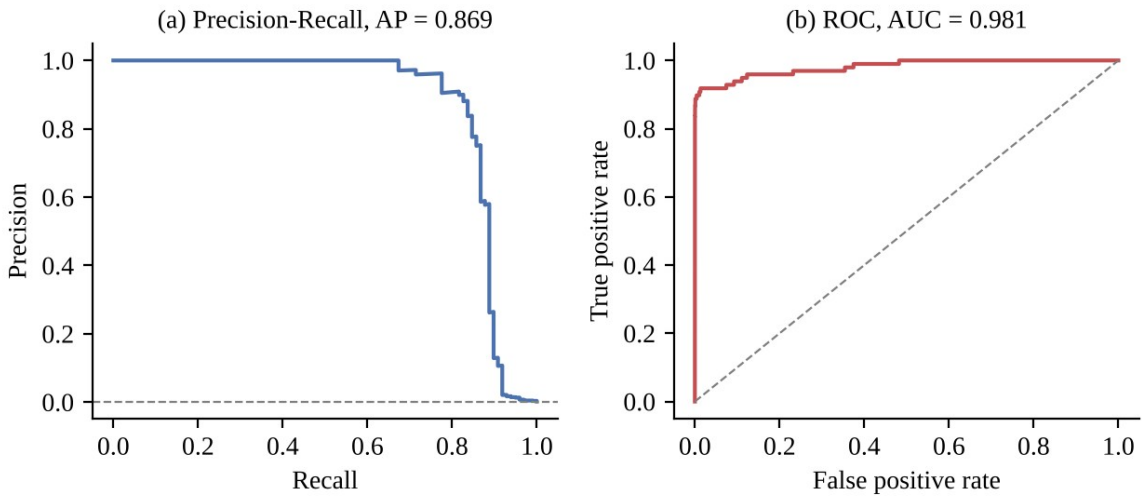


Fig. 7.3 (a) Precision-recall curve and (b) ROC curve on the holdout set

To check whether the model beats simpler alternatives, we trained four baselines on the same split (Table 7.3): logistic regression with balanced class weights, a 200-tree random forest with balanced weights, and XGBoost with default settings, with and without the class weight. The 95% confidence intervals for PR-AUC come from 1,000 bootstrap resamples of the holdout.

Table 7.3 Comparison with baseline models (holdout, 56,962 transactions, 98 frauds)

Model	PR-AUC [95% CI]	ROC-AUC	Precision	Recall	TP	FP
Logistic regression (balanced)	0.719 [0.63, 0.81]	0.972	0.061	0.918	90	1385
Random forest (balanced)	0.858 [0.78, 0.93]	0.947	0.961	0.745	73	3
XGBoost (default)	0.878 [0.81, 0.94]	0.974	0.919	0.806	79	7
XGBoost (default + class weight)	0.880 [0.81, 0.94]	0.968	0.882	0.837	82	11
XGBoost (tuned, ours)	0.869 [0.80, 0.93]	0.981	0.863	0.837	82	13

The three XGBoost models form the best group. Logistic regression has a high ROC-AUC (0.972) but a precision of only 0.061 at the default threshold, with 1,385 false alarms, and the random forest is the most precise model but misses 25 of the 98 frauds. Hyperparameter tuning did not add a measurable gain: the paired bootstrap difference in PR-AUC between the tuned model and default XGBoost with the class weight is −0.011 (95% CI [−0.028, 0.004]). The main gains come from choosing XGBoost and from the class weight, not from the search.

7.3 Effect of the Decision Threshold

The API uses the default threshold of 0.5. Since the model outputs a probability, a bank could move the threshold to match the size of its review team. Table 7.4 shows the trade-off on the holdout. Lowering the threshold to 0.1 catches three more frauds but more than triples the false alarms. Raising it to 0.9 cuts false alarms to 8 but misses two more frauds. The F1-score

is fairly flat between 0.3 and 0.9, so 0.5 is a reasonable middle point, but the choice is really a business decision about what a missed fraud costs compared to an unnecessary review.

Table 7.4 Precision and recall at different thresholds (holdout, 98 frauds)

Threshold τ	TP	FP	FN	Precision	Recall	F1
0.1	85	46	13	0.649	0.867	0.742
0.2	84	25	14	0.771	0.857	0.812
0.3	83	18	15	0.822	0.847	0.834
0.5	82	13	16	0.863	0.837	0.850
0.7	82	11	16	0.882	0.837	0.859
0.9	80	8	18	0.909	0.816	0.860

7.4 Explainability Results

To see which features the model relies on overall, we averaged the absolute SHAP values over a random sample of 5,000 holdout rows (Fig. 7.4). V4 and V14 stand out clearly, followed by V12 and V10. This matches what is commonly reported for this dataset and matches the explanation shown in Section 6.1, where V14, V10 and V12 were the top three reasons for the fraud label. Amount and Time contribute relatively little. This is useful when reading the drift results: drift in V4, V14, V12 or V10 matters much more than drift in a low-importance feature, and the alert description in alert_rules.yml points the on-call person to these features first.

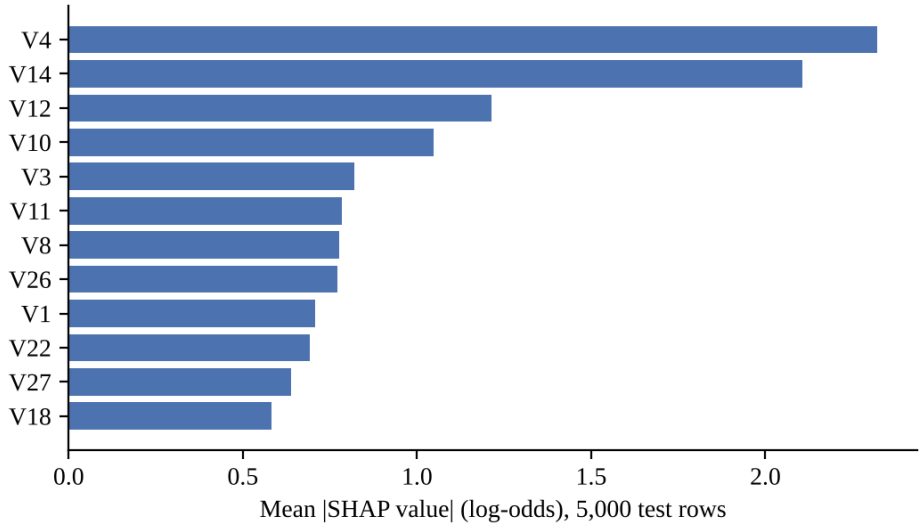


Fig. 7.4 Global feature importance (mean absolute SHAP value)

7.5 Drift Detection Results

Fig. 7.5 compares the per-feature drift scores of the two windows from Section 6.4. In the normal window every feature stayed close to or below the 0.1 threshold; V20 and V21 were just above it (0.104 and 0.101), which is the kind of noise expected from a 300-row sample, and the drift share of 0.069 stayed far below the 0.3 alert level. In the shifted window, all thirteen modified features (twelve PCA components and Amount) scored between 0.78 and 1.63, and five unmodified features (V15, V21, V22, V25 and V28) also crossed 0.1 by a small

margin, giving 18 drifted features in total. The detector therefore separated the two situations clearly. The 10-minute hold on the alert rules adds a second layer of protection against a single noisy window. We repeated the check with the default 500-transaction window: normal traffic flagged 1 of 29 features (s = 0.034) and shifted traffic 15 of 29 (s = 0.517), so the result holds at the window size used in deployment (Table 7.5).

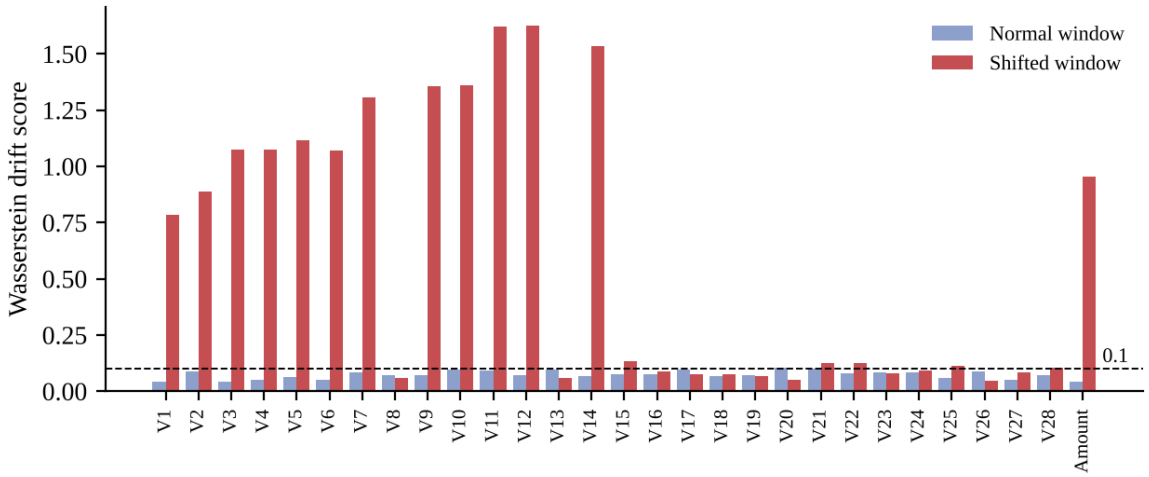


Fig. 7.5 Per-feature drift scores for a normal and a shifted window of 300 transactions

Table 7.5 Drift check summary

Window	Rows	Drifted features	Drift share s	Dataset drift
Normal test traffic	300	2 / 29	0.069	No
Shifted traffic	300	18 / 29	0.621	Yes (alert pending)
Normal test traffic	500	1 / 29	0.034	No
Shifted traffic	500	15 / 29	0.517	Yes

7.6 Latency

We measured latency in a Linux test environment (Python 3.10) with the pinned library versions. Table 7.6 shows the results. Scoring the entire holdout of 56,962 rows in one batch took 0.22 seconds. A single in-process prediction took a median of 8.4 ms, and the SHAP explanation added about 12.5 ms, but only for flagged transactions. Over HTTP, through FastAPI with validation and drift buffering, the median end-to-end time for /prediction_api was 27.5 ms and the 95th percentile was 39.4 ms over 300 requests. This is well within the 100 ms target in Section 3.1.2. The drift computation itself takes a few seconds, mostly to start the worker process and import Evidently, but it runs in the background after the response is sent, so it does not appear in these numbers.

Table 7.6 Measured latency

Operation	Median	95th percentile
Model predict_proba, single row (in process)	8.4 ms	10.8 ms
TreeSHAP explanation, single row	12.5 ms	-
/prediction_api end to end over HTTP	27.5 ms	39.4 ms
Batch scoring of 56,962 rows	0.22 s	-

7.7 Cost Analysis

The software cost of the project is zero, since every component is open source and GitHub Actions is free for public repositories. The compute cost is also small. The whole tuning and training run (100 cross-validation fits plus the final refit) took about 385 seconds on a laptop CPU, the model file is 515 KB, and the serving container needs well under one CPU core at the latencies measured above. In a real deployment, the main costs would be a small virtual machine or a Kubernetes node for the API and the monitoring stack, plus storage for MLflow artifacts and Prometheus data.

A more meaningful cost for this problem is the cost of errors. At the default threshold the model produced 16 false negatives and 13 false positives on the holdout. A false negative means the fraud goes through and the money is lost, while a false positive costs an analyst a few minutes and may annoy a customer. Because missed frauds are usually far more expensive than reviews, a bank with enough reviewers might choose a threshold of 0.2 or 0.3 from Table 7.4, accepting up to about twice as many false alarms (18–25 instead of 13) to catch one or two more frauds out of 98. The system makes this choice easy, since the probability is already returned by the API.

7.8 Discussion and Limitations

Several limitations should be kept in mind. The dataset covers only two days, so it cannot show real seasonal or long-term drift. This is why we demonstrated the drift detector with a controlled shift instead of natural drift. The PCA features hide the original meaning of each variable, so an explanation like "V14 was very negative" is precise but not directly understandable to a business user. In a real bank, the same pipeline would run on named features and the explanations would be much easier to interpret. Input drift is also only an early warning: a change in inputs does not always hurt performance, and performance can drop with no visible input drift. Measuring actual performance over time requires the delayed fraud labels, which this dataset cannot simulate.

Two engineering shortcuts also remain. The API originally asked MLflow for the best run on every request. On the Docker bind mount this took about 4 seconds, because MLflow's file store reads every run's metadata. The API now caches the run ID for 60 seconds and refreshes it in a background thread, which brought requests down to about 50 ms. Pinning a registered model version would remove the lookup entirely. The drift buffer lives in memory, so it is reset when the container restarts and is not shared between replicas if the deployment is scaled out.

CHAPTER 8

CONCLUSION

8.1 Conclusion

We built a complete MLOps pipeline for credit card fraud detection, from versioned data to monitored containers. The model, an XGBoost classifier tuned with Hyperopt using stratified cross-validated PR-AUC, achieves a holdout PR-AUC of 0.869 and catches 82 of 98 frauds with only 13 false alarms out of almost 57,000 transactions. Every training trial is tracked in MLflow, and the API automatically serves the best run. Each fraud-flagged prediction comes with an exact TreeSHAP explanation at a cost of about 12 ms. Input drift is checked every 500 requests by an isolated Evidently worker and reported through Prometheus metrics, alert rules and a Grafana dashboard. In our tests this detector clearly separated normal traffic (2 of 29 features drifted) from shifted traffic (18 of 29). The service is protected by a 16-test suite that runs in CI and during the Docker build, and it can be started locally with one command or deployed to Kubernetes.

For us, the most useful lessons came from the engineering problems rather than the model. These included the dependency conflict that forced the drift worker into its own process, the Docker mount issues that wrote placeholder files back into the repository, and the baseline test that passed or failed depending on which MLflow run happened to be best. Each of these would have caused trouble in production, and dealing with them taught us more about MLOps than tuning the model did.

8.2 Future Work

- Close the loop from drift to retraining, with delayed labels, a champion/challenger comparison on recent data and an automatic rollback.
- Register models in the MLflow Model Registry and serve a pinned version or stage instead of searching runs at request time.
- Route the alerts through Alertmanager to email or Slack, and add performance monitoring once labels become available.
- Move the drift buffer to a shared store such as Redis so that drift is computed across all replicas.
- Add explanations for batch predictions, and test calibrated probabilities and cost-based threshold selection.
- Evaluate the pipeline on a dataset with named features and a longer time span to study natural drift.

CHAPTER 9

REFERENCES

1. Dal Pozzolo, A., Caelen, O., Johnson, R.A. and Bontempi, G., 2015. Calibrating probability with undersampling for unbalanced classification. In *2015 IEEE Symposium Series on Computational Intelligence*, pp.159-166. IEEE.
2. Chen, T. and Guestrin, C., 2016. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp.785-794.
3. Lundberg, S.M. and Lee, S.I., 2017. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, pp.4765-4774.
4. Lundberg, S.M., Erion, G., Chen, H., DeGrave, A., Prutkin, J.M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N. and Lee, S.I., 2020. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1), pp.56-67.
5. Bergstra, J., Yamins, D. and Cox, D., 2013. Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision architectures. In *International Conference on Machine Learning*, pp.115-123. PMLR.
6. Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.F. and Dennison, D., 2015. Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, 28, pp.2503-2511.
7. Kreuzberger, D., Kühl, N. and Hirschl, S., 2023. Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11, pp.31866-31879.
8. Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M. and Bouchachia, A., 2014. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4), pp.1-37.
9. Saito, T. and Rehmsmeier, M., 2015. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3), e0118432.
10. Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C. and Bontempi, G., 2018. Credit card fraud detection: A realistic modeling and a novel learning strategy. *IEEE Transactions on Neural Networks and Learning Systems*, 29(8), pp.3784-3797.
11. Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S.A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M. and Xie, F., 2018. Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4), pp.39-45.
12. Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P., 2002. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, pp.321-357.
13. Breck, E., Cai, S., Nielsen, E., Salib, M. and Sculley, D., 2017. The ML test score: A rubric for ML production readiness and technical debt reduction. In *2017 IEEE*

International Conference on Big Data, pp.1123-1132. IEEE.

14. Rabanser, S., Günnemann, S. and Lipton, Z., 2019. Failing loudly: An empirical study of methods for detecting dataset shift. *Advances in Neural Information Processing Systems*, 32.
15. Bhattacharyya, S., Jha, S., Tharakunnel, K. and Westland, J.C., 2011. Data mining for credit card fraud: A comparative study. *Decision Support Systems*, 50(3), pp.602-613.
16. Paleyes, A., Urma, R.G. and Lawrence, N.D., 2022. Challenges in deploying machine learning: A survey of case studies. *ACM Computing Surveys*, 55(6), pp.1-29.

APPENDIX A – SAMPLE CODE

A.1 Preprocessing transformers (prediction_model/processing/preprocessing.py)

```
from sklearn.base import BaseEstimator,TransformerMixin
from sklearn.preprocessing import RobustScaler
from prediction_model.config import config
import numpy as np


class MeanImputer(BaseEstimator,TransformerMixin):
    def __init__(self,variables=None):
        self.variables = variables

    def fit(self,X,y=None):
        self.mean_dict = {}
        for col in self.variables:
            self.mean_dict[col] = X[col].mean()
        return self

    def transform(self,X):
        X = X.copy()
        for col in self.variables:
            X[col] = X[col].fillna(self.mean_dict[col])
        return X


class RobustScalerTransformer(BaseEstimator,TransformerMixin):
    def __init__(self,variables=None):
        self.variables = variables

    def fit(self,X,y=None):
        self.scaler_ = RobustScaler()
        self.scaler_.fit(X[self.variables])
        return self

    def transform(self,X):
        X = X.copy()
        X[self.variables] = self.scaler_.transform(X[self.variables])
        return X
```

A.2 Single pipeline definition (prediction_model/pipeline.py)

```
def build_pipeline(classifier=None):
    steps = [
        ('MeanImputation', pp.MeanImputer(variables=config.FEATURES)),
        ('RobustScaling',
pp.RobustScalerTransformer(variables=config.ROBUST_SCALE_FEATURES)),
    ]
    if classifier is not None:
        steps.append(('classifier', classifier))
    return Pipeline(steps)
```

A.3 Training and tuning entry point (prediction_model/training_pipeline.py)

```
if __name__ == '__main__':
    mlflow.set_experiment(config.EXPERIMENT_NAME)

    with mlflow.start_run(run_name='fraud_xgb_hyperopt'):
        trials = Trials()
        best = fmin(fn=objective, space=search_space, algo=tpe.suggest,
                    max_evals=config.MAX_EVALS, trials=trials)
        best_params = space_eval(search_space, best)
        print("Best hyperparameters:", best_params)

        # Refit the best config on the full training split and evaluate ONCE on the
        # untouched holdout. Preprocessing is fit inside the pipeline on X_train only,
        # so no information from X_test leaks into scaling/imputation.
        final_pipeline = pipe.build_pipeline(make_classifier(best_params))
        final_pipeline.fit(X_train, y_train)

        proba = final_pipeline.predict_proba(X_test)[:, 1]
        preds = (proba >= 0.5).astype(int)

        metrics = {
            'pr_auc': average_precision_score(y_test, proba),   # primary / model-
selection metric
```

```
            'roc_auc': roc_auc_score(y_test, proba),
            'precision': precision_score(y_test, preds, zero_division=0),
            'recall': recall_score(y_test, preds, zero_division=0),
            'f1': f1_score(y_test, preds, zero_division=0),
        }
        print("Holdout metrics:", {k: round(v, 4) for k, v in metrics.items()})

        mlflow.log_params(best_params)
        mlflow.log_metrics(metrics)
        mlflow.sklearn.log_model(final_pipeline, config.MODEL_NAME.lstrip('/'))
```

A.4 Cached model loading and explained prediction (prediction_model/predict.py)

```
_RUN_ID_CACHE = {"run_id": None, "resolved_at": 0.0, "refresh": None}
_RUN_ID_LOCK = threading.Lock()

def _query_best_run_id():
    experiment = mlflow.get_experiment_by_name(config.EXPERIMENT_NAME)
    runs_df = mlflow.search_runs(experiment_ids=experiment.experiment_id,
                                 order_by=['metrics.pr_auc DESC'])
    return runs_df.iloc[0]['run_id']

def _refresh_run_id():
    try:
        run_id = _query_best_run_id()
        with _RUN_ID_LOCK:
            _RUN_ID_CACHE.update(run_id=run_id, resolved_at=time.monotonic())
    except Exception:  # noqa: BLE001 - keep serving the cached run; retry after the next
TTL
        with _RUN_ID_LOCK:
            _RUN_ID_CACHE["resolved_at"] = time.monotonic()
    finally:
        with _RUN_ID_LOCK:
            _RUN_ID_CACHE["refresh"] = None

def _resolve_best_run_id():
    if _RUN_ID_CACHE["run_id"] is None:
        run_id = _query_best_run_id()
        with _RUN_ID_LOCK:
            _RUN_ID_CACHE.update(run_id=run_id, resolved_at=time.monotonic())
        return run_id
    with _RUN_ID_LOCK:
        expired = time.monotonic() - _RUN_ID_CACHE["resolved_at"] >=
config.RUN_ID_CACHE_SECONDS
        if expired and _RUN_ID_CACHE["refresh"] is None:
            _RUN_ID_CACHE["refresh"] = threading.Thread(target=_refresh_run_id,
daemon=True)
            _RUN_ID_CACHE["refresh"].start()
        return _RUN_ID_CACHE["run_id"]

def _get_model_and_explainer():
    run_id = _resolve_best_run_id()
    if run_id != _CACHE["run_id"] or _CACHE["model"] is None:
        model = mlflow.sklearn.load_model('runs:/' + run_id + config.MODEL_NAME)
        _CACHE.update(run_id=run_id, model=model,
explainer=explain.build_explainer(model))
    return _CACHE["model"], _CACHE["explainer"]

def generate_prediction_with_explanation(data_input):
    data = pd.DataFrame(data_input)[config.FEATURES]
    model, explainer = _get_model_and_explainer()

    fraud_probability = float(model.predict_proba(data)[:, 1][0])
    prediction = 'fraud' if model.predict(data)[0] == 1 else 'legit'

    explanation = None
    if prediction == 'fraud' or not config.EXPLAIN_ONLY_FLAGGED:
        explanation = explain.explain_row(model, explainer, data, config.SHAP_TOP_N)

    return {"prediction": prediction,
            "fraud_probability": fraud_probability,
            "explanation": explanation}
```

A.5 Local SHAP explanation (prediction_model/explain.py)

```
def explain_row(model, explainer, X_row, top_n):
    X_transformed = model[:-1].transform(X_row)
    shap_row = _positive_class_shap(explainer.shap_values(X_transformed))[0]
```

```
    raw = X_row.iloc[0]
    contributions = [
        {
            'feature': feature,
            'value': float(raw[feature]),          # raw submitted value (human-readable)
            'shap_value': float(shap_row[i]),      # contribution in log-odds/margin
space
            'direction': 'increases_fraud_risk' if shap_row[i] > 0 else
'decreases_fraud_risk',
        }
        for i, feature in enumerate(X_transformed.columns)
    ]
    contributions.sort(key=lambda c: abs(c['shap_value']), reverse=True)

    return {
        'base_value': _base_value(explainer.expected_value),
        'top_features': contributions[:top_n],
    }
```

A.6 Drift buffer and worker invocation (prediction_model/monitoring/drift.py)

```
def record_inference(row):
    global _COUNT
    monitored = {f: row[f] for f in config.DRIFT_FEATURES if f in row}
    with _LOCK:
        _BUFFER.append(monitored)
        _COUNT += 1
        should_fire = (len(_BUFFER) >= config.DRIFT_WINDOW
                       and _COUNT % config.DRIFT_WINDOW == 0)
    return should_fire


def _snapshot_current():
    with _LOCK:
        if not _BUFFER:
            return None
        return pd.DataFrame(list(_BUFFER))


def _require_reference():
    if not os.path.exists(config.DRIFT_REFERENCE_FILE):
        raise FileNotFoundError(
            f"Drift reference not found at {config.DRIFT_REFERENCE_FILE}. "
            f"Generate it with: python -m prediction_model.monitoring.make_reference")


def compute_drift(current_df):
    _require_reference()
    with tempfile.TemporaryDirectory() as tmp:
        current_csv = os.path.join(tmp, "current.csv")
        out_json = os.path.join(tmp, "drift.json")
        current_df.to_csv(current_csv, index=False)

        proc = subprocess.run(
            [sys.executable, _WORKER,
             "--current", current_csv,
             "--reference", config.DRIFT_REFERENCE_FILE,
             "--out", out_json,
             "--alert-share", str(config.DRIFT_ALERT_SHARE)],
            capture_output=True, text=True,
        )
        if proc.returncode != 0 or not os.path.exists(out_json):
            raise RuntimeError(
                f"drift worker failed (rc={proc.returncode}): {proc.stderr[-800:]}")
        with open(out_json, encoding="utf-8") as fh:
            return json.load(fh)
```

A.7 Prediction endpoint (main.py)

```
def _record_and_maybe_check(data, background_tasks):
    if not config.DRIFT_ENABLED:
        return
    if drift.record_inference(data):
        background_tasks.add_task(drift.safe_run_drift_check)

@app.post("/prediction_api", response_model=PredictionResponse)
def predict(transaction: TransactionFeatures, background_tasks: BackgroundTasks):
    data = transaction.model_dump()
    result = generate_prediction_with_explanation([data])
```

```
    _record_and_maybe_check(data, background_tasks)
    return {
        "status": _label(result["prediction"]),
        "fraud_probability": result["fraud_probability"],
        "explanation": result["explanation"],   # None unless fraud-flagged
    }
```